

Get All Combinations of Balanced Parentheses

A string has *balanced parentheses* if every opening parenthesis is followed by exactly one closing parenthesis. For example, `'()()'` and `'(())'` are strings of two balanced parentheses pairs, but `'()()'` and `'(())'` are not balanced. These strings are also called *Dyck words*, after mathematician Walther von Dyck.

A common coding interview question is to write a recursive function that, given the number of pairs of parentheses, produces all possible combinations of balanced parentheses. For example, a `getBalancedParens(3)` call should return `['((()))', '(()())', '()()()', '()(())', '()()()']`. Note that calling `getBalancedParens(n)` returns strings that are $2n$ characters in length, since each string consists of n pairs of parentheses.

We could try to solve this problem by finding all permutations of the pairs of parenthesis characters, but that would result in both balanced and unbalanced parentheses strings. Even if we filtered out the invalid strings later, $2n!$ permutations exist for n pairs of parentheses. That algorithm is far too slow to be practical.

Instead, we can implement a recursive function to generate all strings of balanced parentheses. Our `getBalancedParens()` function takes an integer of the number of pairs of parentheses and returns a list of balanced parentheses strings. The function builds these strings by adding either an opening or closing parenthesis. An opening parenthesis can be added only if opening parentheses remain to be used. A closing parenthesis can be added only if more opening parentheses have been added than closing parentheses so far.

We'll track the number of opening and closing parentheses remaining to be used with function parameters named `openRem` and `closeRem`. The string currently being built is another function parameter named `current`, which serves a similar purpose as the `prefix` parameter in the `permutationsWithRepetition` program. The first base case occurs when `openRem` and `closeRem` are both 0 and no more parentheses remain to be added to the `current` string. The second base case happens after the two recursive